

Introduction to Graphs

For additional reading and resources, you can refer to <https://jeffe.cs.illinois.edu/teaching/algorithms/book/05-graphs.pdf>

What is a Graph?

A graph consists of vertices and edges. Vertices may also be known as nodes. More formally, a simple graph is a pair of sets (V, E) , where V is an arbitrary non-empty finite set, whose elements are called vertices, and E is a set of pairs of elements of V , which we call edges.

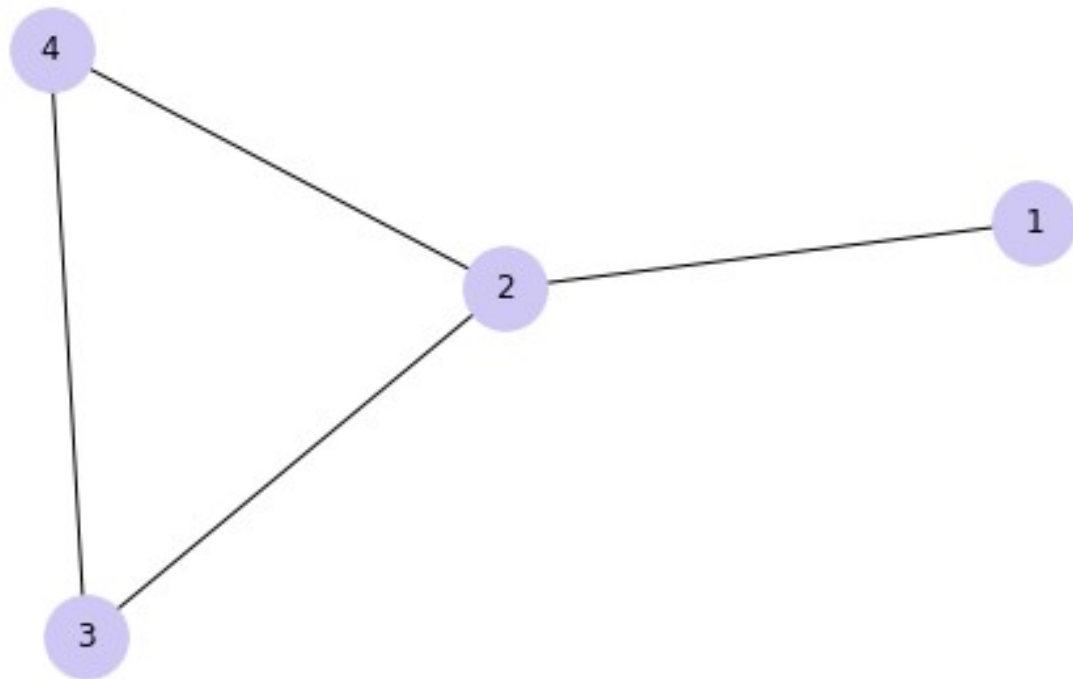
Examples

- social network: each user corresponds to a vertex, two users that are friends are connected by an edge
- flight network: each airport corresponds to a vertex, two airports with direct flights are connected by an edge

Below is an example of a graph. We will use the `networkx` library to draw graphs in Python:

```
import networkx as nx

G = nx.Graph()
G.add_edges_from([(1, 2), (2, 4), (3, 2), (3,4)])
nx.draw(G, with_labels=True, node_size=1000, node_color='#cfc8f4')
```



In the above graph, what is the set V?
What is the set E?

From the above graph, we see that edges connect one vertex to another vertex. In our graph example, 1, 2, 3, and 4 represent nodes or vertices of a graph.

There are two types of graphs: undirected and directed.

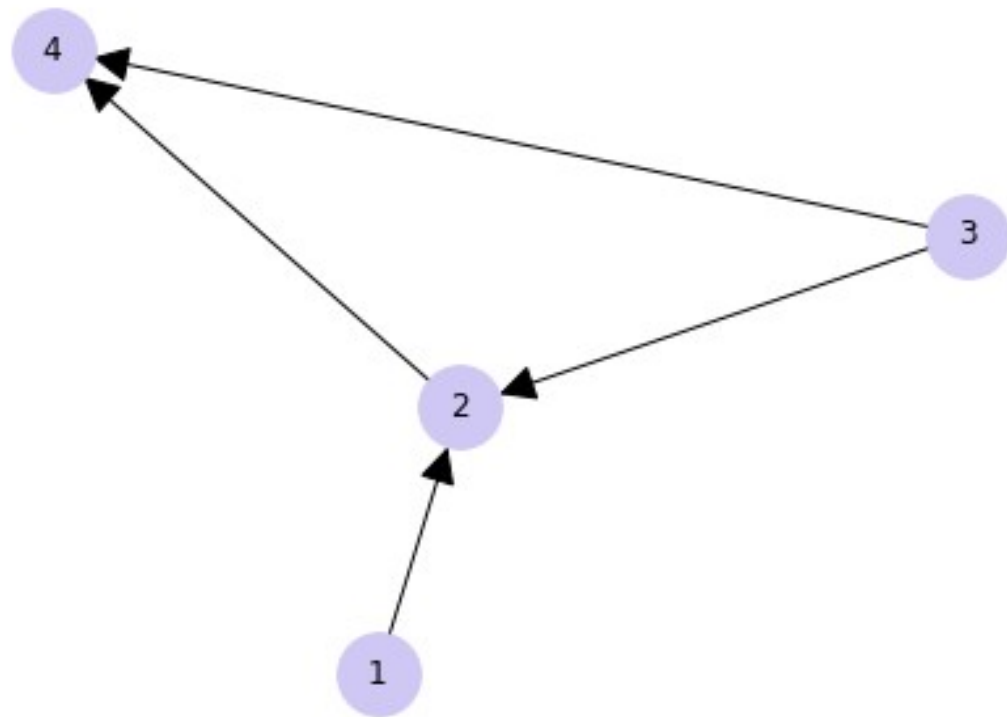
In an **undirected** graph, the edges are unordered pairs. This means when an edge joins vertex A and vertex B, this edge indicates a two-way relationship between A and B where the edge can be traversed in both directions. The graph example we saw above is an example of an undirected graph.

In a **directed** graph, the edges are ordered pairs of vertices. This means when an edge joins vertex A to vertex B, this edge has a direction, usually indicated by an arrow. If there is only an edge from A to B, then the edge can only be traversed from A to B and not in the reverse.

Below, is an example of a directed graph.

```
import networkx as nx

G = nx.DiGraph()
G.add_edges_from([(1, 2), (2, 4), (3, 2), (3,4)])
nx.draw(G, with_labels=True, node_size=1000, node_color='#cfc8f4',
arrowsize=30)
```



We call a graph a **simple graph** if there are no self-loops or parallel edges.

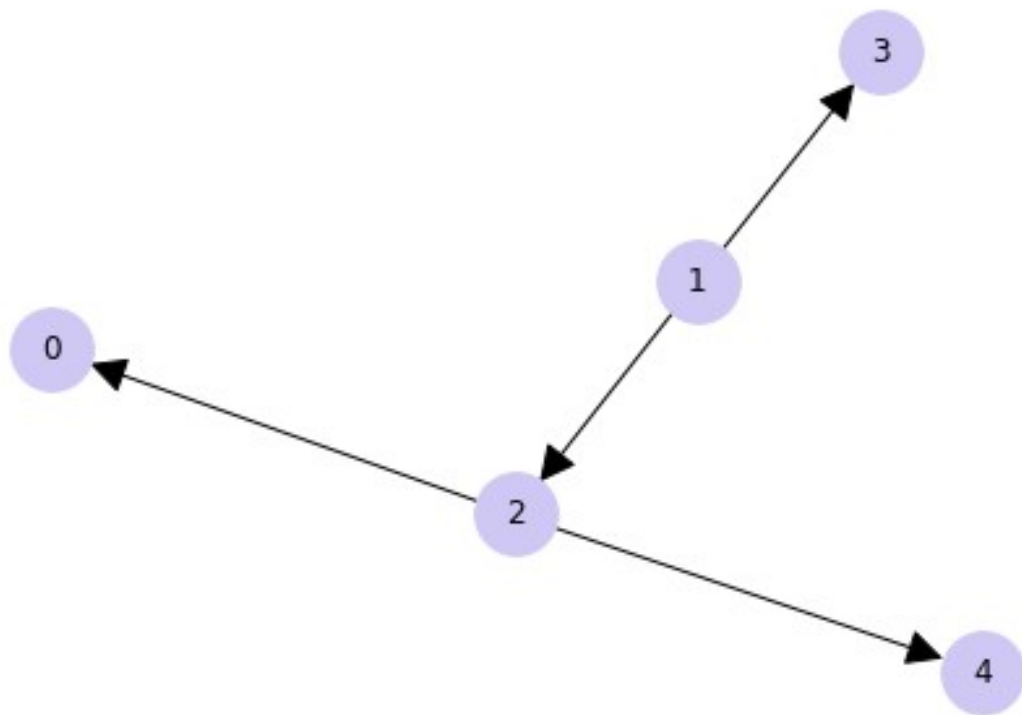
Common Graph Terminology

Let's consider an edge in an undirected graph between vertex u and vertex v . We call u and v neighbors, and vice versa. We can say that u and v are adjacent. The degree of a node is the number of neighbors.

For a directed graph, we distinguish two kinds of neighbors. For any directed edge from u to v , we call u a predecessor of v and we call v a successor of u . The in-degree of a vertex is its number of predecessors and the out-degree is its number of successors.

```
def draw(graph):
    g = nx.DiGraph(graph)
    nx.draw(g, with_labels=True, node_size=1000, node_color='#cfc8f4',
            arrowsize=30)

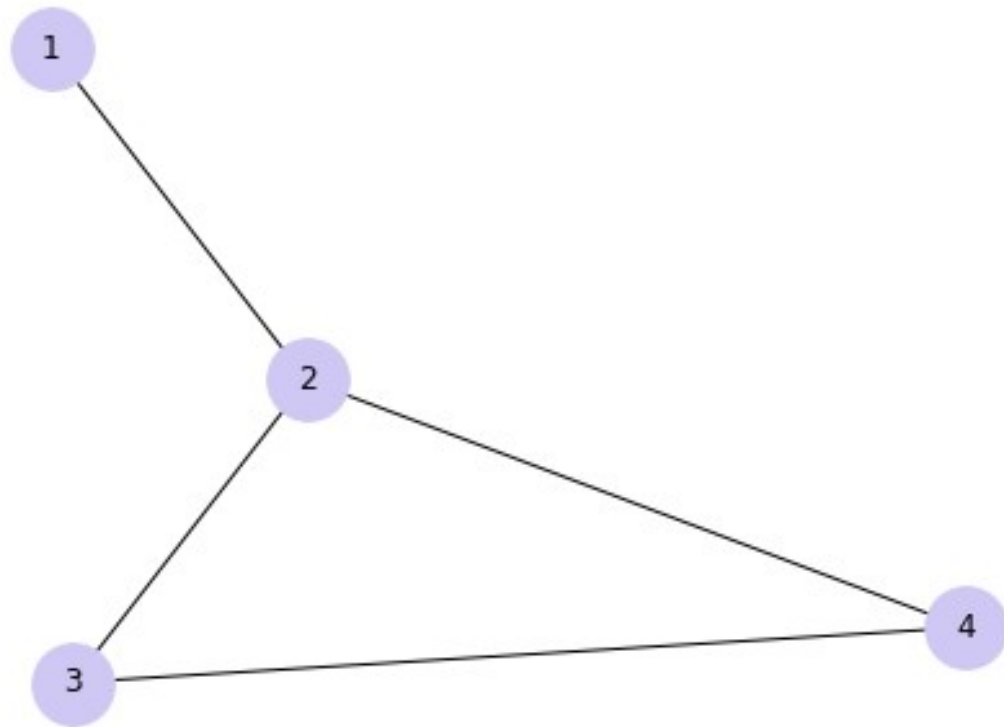
g = [[1, 2], [1, 3], [2, 0], [2, 4]]
draw(g)
```



Consider the graph above. With your neighbor, try to answer the following questions:

1. What is the in-degree of 2?
2. What is the out-degree of 1?
3. Is 4 a predecessor of 2?

```
G = nx.Graph()
G.add_edges_from([(1, 2), (2, 4), (3, 2), (3, 4)])
nx.draw(G, with_labels=True, node_size=1000, node_color='#cfc8f4')
```



A walk in an undirected graph, G , is a sequence of vertices, where each adjacent pair of vertices are adjacent in G . You can also think of this as a sequence of edges. If we consider the undirected graph above, $2 \rightarrow 3 \rightarrow 4$ is an example of a walk.

What are some other examples of a walk for the graph, G ?

A walk becomes a path if it visits each vertex at most once. If we revisit the undirected graph example, the walk $2 \rightarrow 3 \rightarrow 4$ is also a path.

For any two vertices u and v in a graph, G , we say that v is reachable from u if G contains a walk between u and v . An undirected graph is connected if every vertex is reachable from every other vertex. The undirected graph above is connected.

A walk is closed if it starts and ends at the same vertex.

Talk with your neighbor. Can you come up with a closed walk from the graph above?

For a directed graph, the definitions are slightly different. For example, we have a directed walk which is a sequence of vertices that follows directed edges. Vertex v is reachable from vertex u in a directed graph G if and only if G contains a directed walk (and therefore a directed path) from u to v . A directed graph is strongly connected if every vertex is reachable from every other vertex.

How can we represent graphs in code?

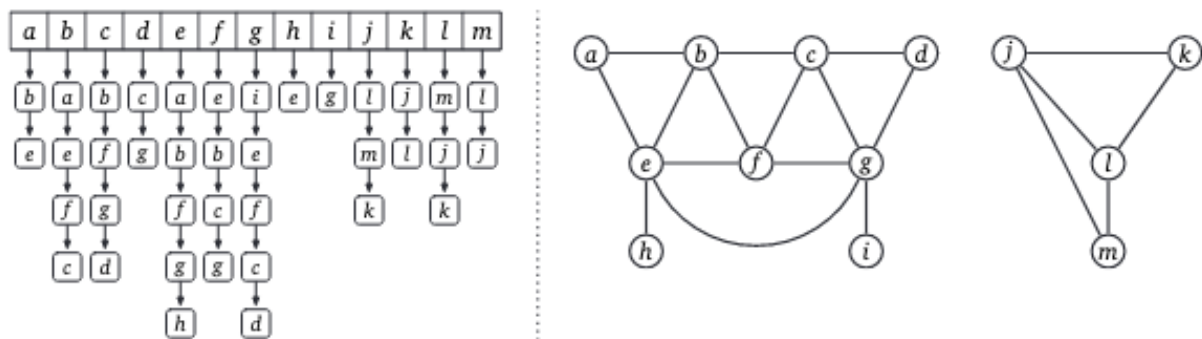
The most common way to visualize graphs is to draw them, similar to what we did above. However, this doesn't work well for translating a graph to something that can be used by code. In practice, we usually represent a graph with one of two data structures:

1. Adjacency Lists
2. Adjacency Matrices

Adjacency List

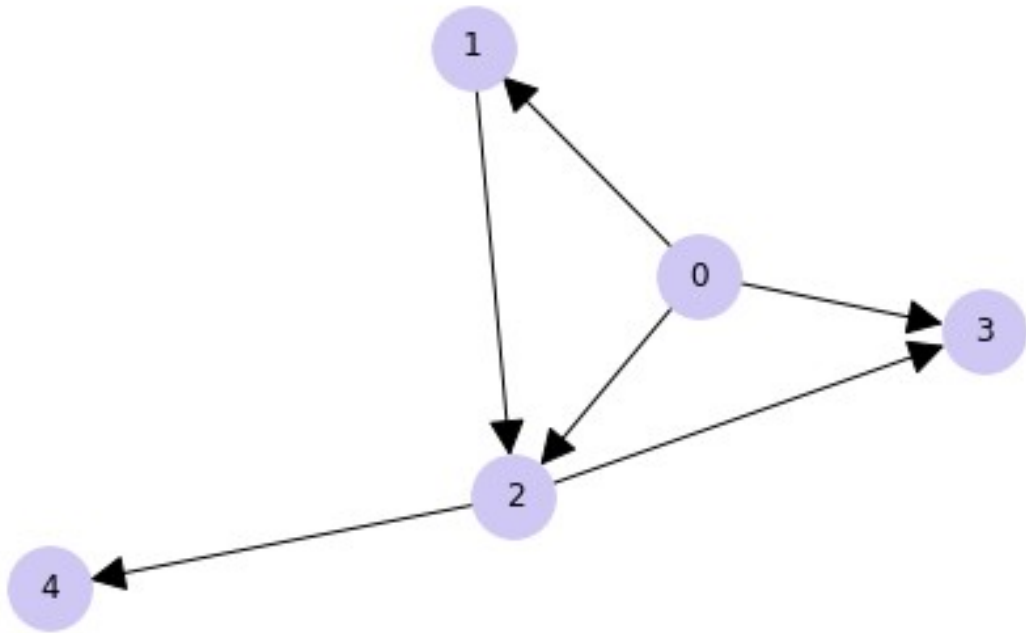
This is one of the most common data structures for storing graphs. An adjacency list is an array of lists, each containing the neighbors of one of the vertices (or the out-neighbors if the graph is directed). For undirected graphs, we have to store each edge twice, once for each vertex. For directed graphs, each edge is stored only once for the vertex that it comes from. For both types of graphs, the overall space required for an adjacency list is $O(V + E)$.

The standard implementation of an adjacency list uses something called a linked list. We haven't covered a linked list yet, but below is an example to help illustrate what a linked list might look like:



To make code implementation a bit simpler for us, we will represent our adjacency list using a dictionary which we will be calling the **adjacency dictionary**. Below is an example of an adjacency dictionary:

```
test_graph = {0: [1, 2, 3], 1: [2], 2: [4, 3]}
draw(test_graph)
```



Adjacency Matrix

The second standard data structure for graphs is the adjacency matrix. The adjacency matrix of a graph, G , is a $V \times V$ matrix of 0s and 1s, normally represented by a two-dimensional array $A[1 \dots V, 1 \dots V]$, where each entry indicates whether a particular edge is present in G . 1 represents the edge exists and 0 represents that the edge does not exist.

	v1	v2	v3	v4
v1	0	1	0	0
v2	0	0	1	0
v3	1	0	0	1
v4	0	0	0	0

For the labs and this class, we will focus only on the adjacency dictionary when trying to implement graphs for now.

Review if there is time

Let's review a few key concepts from the last few weeks. Some of you may have already gone over your quizzes, so some of this may look familiar to you.

Adding two lists together

```
# y sum question from quiz
arr = [[1], [2, 3], [5, 6, 1]]
```

```
def ysum(lst):
    sum = 0
    for i in lst:
        sum += i
    return sum

def fun(lst):
    res = []
    for i in lst:
        res += [ysum(i)] # what does this output
    return res

fun(arr)

[1, 5, 12]
```

Using loops and working with dictionaries

```
# count characters question "addis" => {"a": 1, "d": 2, "i": 1, "s": 1}

def count_characters(str):
    res = {}
    for i in str:
        # check if i exists in res, if it doesnot add it
        if i not in res:
            res[i] = 1
        else:
            res[i] = res[i] + 1
        # add to existing value, if it does
    return res

count_characters("addis")

{'a': 1, 'd': 2, 'i': 1, 's': 1}
```

Working with offsets

```
# question to print from 1 to n or offset other values
def sum_of_digits(num):
    sum = 0
    # input: 28394 % 10 => 4
    # 2839 % 10 => 9
    while(num > 0):
        sum += num % 10
        num //= 10
    return sum

sum_of_digits(28394)
```

